SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
   <head>
      <title>Company</title>
   </head>
   <body>
      <div>
         <h1>Welcome
         <p>About us
      </div>
   </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
   <input type="text" name="username">
   <input type="password" name="password">
   <button type="button">Login</button>
</form>
```

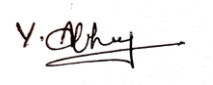
Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I Y. Abhay-192411219 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Case Study 1: Online Symposium Portal

Given HTML:

```
<form action="/register" method="POST">[Text Wrapping Break]  <input type="text" name="name">[Text Wrapping Break]  <input type="email" name="email">[Text Wrapping Break]  <button>Submit</button>[Text Wrapping Break]</form>
```

1. Semantic elements missing in the structure

The form is a bare block with no page context, labelling, or grouping. The following HTML5 semantic elements are missing:

- <header> / <main> / <section> — to place the registration form within the page's document outline.
- <label> — every input lacks an associated <label>, so the field's purpose is not programmatically tied to it.
- <fieldset> and <legend> — to group the related registration fields under a titled group.
- <h1>/<h2> heading — no heading identifies the form as "Symposium Registration".
- required / placeholder / autocomplete attributes — not strictly elements, but their absence weakens the semantics of the input fields.

2. Standards-compliant, corrected structure

```
<!DOCTYPE html>[Text Wrapping Break]<html lang="en">[Text Wrapping Break]<head>[Text Wrapping Break]<meta charset="UTF-8">[Text Wrapping Break]  <meta name="viewport" content="width=device-width, initial-scale=1.0">[Text Wrapping Break]  <title>Symposium Registration</title>[Text Wrapping Break]</head>[Text Wrapping Break]<body>[Text Wrapping Break]  <header>[Text Wrapping Break]  <h1>National Symposium Registration</h1>[Text Wrapping Break]  </header>[Text Wrapping Break]  <main>[Text Wrapping Break]  <section>[Text Wrapping Break]  <form action="/register" method="POST">[Text Wrapping Break]  <fieldset>[Text Wrapping Break]  <legend>Participant Details</legend>[Text Wrapping Break]  <label for="name">Full Name</label>[Text Wrapping Break]  <input type="text" id="name" name="name"[Text Wrapping Break]  required autocomplete="name">[Text Wrapping Break]  <label for="email">Email Address</label>[Text Wrapping Break]  <input type="email" id="email" name="email"[Text Wrapping Break]  required autocomplete="email">[Text Wrapping Break]  <button type="submit">Submit</button>[Text Wrapping Break]  </fieldset>[Text Wrapping Break]  </form>[Text Wrapping Break]  </section>[Text Wrapping Break]  </main>[Text Wrapping Break]  <footer>[Text Wrapping Break]  <p>&copy; 2026 College Symposium Committee</p>[Text Wrapping Break]  </footer>[Text Wrapping Break]</body>[Text Wrapping Break]</html>
```

3. HTTP request generated on submission

Because method="POST" and action="/register", the browser packages the two field values as the message body (not the URL) and sends:

```
POST /register HTTP/1.1[Text Wrapping Break]Host: www.symposium-college.edu[Text Wrapping Break]Content-Type: application/x-www-form-urlencoded[Text Wrapping Break]Content-Length: 34[Text Wrapping Break]Connection: keep-alive[Text Wrapping Break]name=Deepak+V&email=deepak%40mail.com
```

Key points: the request line uses the POST method and the form's action path; the Content-Type header tells the server how to decode the body (URL-encoded key=value pairs, separated by &, with spaces encoded as + and special characters percent-encoded); and the actual data is sent in the request body rather than appended to the URL, which keeps it out of browser history and server logs.

4. HTTP response cycle from server to browser

- The server receives the POST request and its embedded form data.
- It validates and processes the data (e.g., inserts the registration into a database).
- It constructs an HTTP response with a status line, headers, and a body, for example:

```
HTTP/1.1 302 Found[Text Wrapping Break]Location: /register/success[Text Wrapping Break]Set-Cookie: session_id=abc123; HttpOnly[Text Wrapping Break]
```

- A 200 OK would return the resulting HTML directly; a 302 Found (as above) redirects the browser to a confirmation page — the Post/Redirect/Get pattern, which prevents duplicate submissions if the user refreshes.
- The browser follows the redirect (or renders the returned HTML), updates the DOM, and displays the confirmation page to the student.

5. Usability and accessibility improvements

- Associate every input with a <label> (via for/id) so screen readers announce the field's purpose.
- Add inline validation messages using aria-describedby and the :invalid CSS pseudo-class.
- Use appropriate input types (email, tel) so mobile devices show the correct keyboard.
- Provide clear focus indicators and a logical tab order for keyboard-only users.
- Show a loading state and disable the submit button after the first click to prevent duplicate submissions.
- Add helpful placeholder text and error summaries at the top of the form for users with cognitive or visual impairments.

Case Study 2: Cross-Browser Compatibility Issue

Given HTML:

```
<html>[Text Wrapping Break]  <head>[Text Wrapping Break]    <title>Company</title>[Text Wrapping Break]
</head>[Text Wrapping Break]  <body>[Text Wrapping Break]    <div>[Text Wrapping Break]
<h1>Welcome[Text Wrapping Break]      <p>About us[Text Wrapping Break]    </div>[Text Wrapping Break]
</body>[Text Wrapping Break]</html>
```

1. Improper nesting and missing closing tags

- No <!DOCTYPE html> declaration — the browser cannot be certain it should render in Standards mode.
- <html> is missing a lang attribute.
- <h1>Welcome is not closed with </h1>.
- About us is not closed with .
- The is closed after the unclosed <h1> and , so the parser must infer where each element actually ends — this is invalid nesting, since an unclosed <h1> cannot legally contain the following .
- No <meta charset> is declared.

2. How different browsers may interpret this structure

All modern browsers implement the HTML5 parsing algorithm's error-recovery rules, so they will auto-close the missing tags — but the exact point at which each engine decides to close a tag can differ subtly between engines (Blink/Chromium, Gecko/Firefox, WebKit/Safari), especially when CSS relies on sibling or descendant selectors.

- Without a DOCTYPE, older or non-standard browsers may render the page in Quirks Mode instead of Standards Mode, changing how box-sizing, margins, and table layout are calculated.

- Because the intended nesting (heading and paragraph both inside the div) is ambiguous, the browser's auto-generated DOM tree may not match what the developer intended, causing CSS selectors like div > p to unexpectedly match or fail to match across browsers.
- The missing charset can cause encoding-dependent characters to render differently depending on the browser's default encoding guess.

3. Missing semantic elements

- <header> — to wrap the site branding/navigation area.
- <main> — to identify the primary content region of the page.
- <section> or <article> — to group the "About us" content meaningfully.
- <nav> — if navigation links exist elsewhere on the real page.
- <footer> — for closing/company information.

4. Corrected, W3C-compliant HTML structure

```
<!DOCTYPE html>[Text Wrapping Break]<html lang="en">[Text Wrapping Break]<head>[Text Wrapping Break]
<meta charset="UTF-8">[Text Wrapping Break]  <meta name="viewport" content="width=device-width,
initial-scale=1.0">[Text Wrapping Break]  <title>Company</title>[Text Wrapping Break]</head>[Text Wrapping
Break]<body>[Text Wrapping Break]  <header>[Text Wrapping Break]    <h1>Welcome</h1>[Text Wrapping
Break]  </header>[Text Wrapping Break]  <main>[Text Wrapping Break]    <section>[Text Wrapping Break]
<h2>About Us</h2>[Text Wrapping Break]      <p>About us</p>[Text Wrapping Break]    </section>[Text
Wrapping Break]  </main>[Text Wrapping Break]  <footer>[Text Wrapping Break]    <p>&copy; 2026 Company
Name</p>[Text Wrapping Break]  </footer>[Text Wrapping Break]</body>[Text Wrapping Break]</html>
```

5. Techniques to ensure cross-browser compatibility

- Always start documents with <!DOCTYPE html> to force Standards mode in every browser.
- Validate markup with the W3C Markup Validator so no tag is left unclosed or improperly nested.
- Use a CSS reset or normalize.css to remove browser-specific default styling differences.
- Use feature detection (e.g., the @supports CSS rule, or a library such as Modernizr) instead of browser-sniffing.
- Test on multiple real browsers/devices or a service like BrowserStack, and use vendor-prefixed properties where a CSS feature is not yet universally supported.
- Use autoprefixer / Babel in the build pipeline so CSS and JavaScript are automatically adjusted for target browser versions.

Case Study 3: Form Submission Failure

Given HTML:

```
<form action="/submit" method="GET">[Text Wrapping Break]  <input type="text"
name="username">[Text Wrapping Break]  <input type="password" name="password">[Text Wrapping
Break]  <button type="button">Login</button>[Text Wrapping Break]</form>
```

1. Issues in the form configuration

- The <button> has type="button", not type="submit" — a plain button element does nothing on click unless JavaScript is attached to it; it will never submit the form.
- method="GET" is used for a login form, which appends the username and, critically, the password to the URL query string in plain text.
- Login credentials should never travel over GET — they end up in the browser history, server access logs, and any proxy logs along the path.

- There is no HTTPS/encryption context indicated, and no CSRF protection token is present.

2. Why the HTTP request is not triggered

An HTML form is only submitted automatically when the browser's default submit behaviour fires — that happens when a <button> or <input> has type="submit" (or is the implicit default type inside a form) and is clicked, or when Enter is pressed in a text field. Because the button here is explicitly type="button", the browser treats it as a plain, inert button with no built-in form-submission behaviour. No 'click' handler has been attached in JavaScript either, so clicking it does nothing — the browser never constructs or sends the GET request to /submit.

3. Role of the GET method in this scenario

GET is designed for retrieving/requesting data, not submitting sensitive data: it encodes all form fields as a query string appended to the URL, e.g.

```
GET /submit?username=deepak&password=mypassword123 HTTP/1.1[Text Wrapping Break]Host: www.example.com
```

Even if the button type were fixed, GET is fundamentally the wrong method here — the password would be visible in the address bar, cached by the browser, stored in server logs, and potentially leaked via the Referer header to any third-party resources loaded on the next page.

4. Corrected implementation

```
<form action="/submit" method="POST">[Text Wrapping Break]  <label for="username">Username</label>[Text Wrapping Break]  <input type="text" id="username" name="username" required>[Text Wrapping Break]  <label for="password">Password</label>[Text Wrapping Break]  <input type="password" id="password" name="password" required>[Text Wrapping Break]<button type="submit">Login</button>[Text Wrapping Break]</form>
```

The two fixes are: change the button to type="submit" so the browser's native submission behaviour fires, and change the form method to POST so credentials travel in the request body instead of the URL.

5. Improvements for secure data transmission

- Serve the login page and endpoint only over HTTPS (TLS) so the POST body is encrypted in transit.
- Never store or transmit plaintext passwords server-side — hash them with a salted algorithm such as bcrypt or Argon2 upon receipt.
- Add CSRF protection (a per-session token included in the form and validated server-side).
- Set autocomplete="current-password" and consider disabling browser autofill caching on shared machines.
- Implement rate limiting / account lockout on repeated failed attempts to resist brute-force attacks.
- Use HttpOnly, Secure, and SameSite cookie attributes for the resulting session cookie.